

PFLICHTENHEFT

CampusConnect

Webbasierte Plattform zur Organisation von Lerngruppen und
Hochschulprojekten

Dokumentstatus	Beispiel / Testprojekt
Version	1.0
Stand	22.09.2026
Zweck	Upload- und Funktionstest in SOPHub

*Hinweis: Dieses Dokument beschreibt ein fiktives Softwareprojekt und dient ausschließlich als
realistisches Beispieldokument für Testzwecke.*

Inhaltsverzeichnis

1. Einleitung
2. Systemübersicht
3. Technische Architektur
4. Funktionale Umsetzung
5. Datenmodell
6. Benutzeroberfläche
7. Nichtfunktionale Umsetzung
8. Schnittstellen und API
9. Fehlerbehandlung und Validierung
10. Projektplan
11. Testkonzept
12. Abnahmekriterien
13. Deployment und Betrieb
14. Qualitätssicherung
15. Glossar

1. Einleitung

1.1 Ziel des Dokuments

Dieses Pflichtenheft konkretisiert die im Lastenheft beschriebenen Anforderungen für CampusConnect und beschreibt, wie die Funktionen technisch und organisatorisch umgesetzt werden sollen. Es bildet die Arbeitsgrundlage für Entwicklung, Integration, Test und Abnahme.

1.2 Projektziel

CampusConnect wird als webbasierte Client-Server-Anwendung umgesetzt. Die Anwendung unterstützt Projektübersicht, Projekterstellung, Team- und Einladungsprozesse, Dokumentenverwaltung sowie rollenabhängige Zugriffe. Die technische Struktur soll eine klare Trennung zwischen Benutzeroberfläche, Geschäftslogik und Datenhaltung gewährleisten.

1.3 Bezug zum Lastenheft

Die Muss- und Soll-Anforderungen des Lastenhefts werden in technische Komponenten, Datenstrukturen, Schnittstellen und Testfälle überführt. Abweichungen sind zu dokumentieren und im Projektteam abzustimmen.

2. Systemübersicht

Das System besteht aus Frontend, Backend, relationaler Datenbank und Dateispeicher. Optional kann ein externer Authentifizierungsdienst angebunden werden. Der Zugriff erfolgt über einen Browser; eine lokale Installation auf dem Endgerät ist nicht erforderlich.

Komponente	Aufgabe
Frontend	Darstellung, Navigation, Formulare, clientseitige Validierung und Kommunikation mit der REST-API.
Backend	Geschäftslogik, Rechteprüfung, Validierung, Projekt- und Dokumentenverwaltung sowie Bereitstellung der REST-Endpunkte.
Datenbank	Persistente Speicherung strukturierter Daten und Beziehungen.
Dateispeicher	Speicherung hochgeladener Projektdokumente.
Authentifizierung	Prüfung von Zugangsdaten und Ermittlung des angemeldeten Nutzers.

2.1 Nutzergruppen

- Studierende: Projekte suchen, anlegen, beitreten, Einladungen bearbeiten und Dokumente verwenden.
- Lehrende: betreute Projekte einsehen und projektbezogene Informationen abrufen.
- Administratoren: Rollen, Berechtigungen und problematische Inhalte verwalten.

3. Technische Architektur

3.1 Frontend

Das Frontend wird als komponentenbasierte Single-Page-Anwendung mit React umgesetzt. Wiederverwendbare Komponenten werden für Navigation, Formulare, Projektkarten, Statusanzeigen und Dialoge verwendet. Die Navigation erfolgt über clientseitiges Routing.

- Darstellung der Seiten Dashboard, Projekte, Projektdetails, Projektanlage, Einladungen und Profil.
- Formularvalidierung vor dem Absenden.
- Abruf und Übermittlung von Daten über HTTP-Anfragen.
- Darstellung von Lade-, Erfolgs- und Fehlerzuständen.
- Rollenabhängige Anzeige von Aktionen.

3.2 Backend

Das Backend wird mit Java und Spring Boot umgesetzt. Controller stellen REST-Endpunkte bereit, Service-Klassen kapseln die Geschäftslogik und Repository-Komponenten übernehmen den Datenzugriff. Datenobjekte für externe Schnittstellen werden von Persistenzobjekten getrennt, sofern dies zur Wartbarkeit erforderlich ist.

- Authentifizierung und Autorisierung.
- Projekt- und Teamverwaltung.
- Einladungslogik.
- Dokumentenmetadaten und Dateizugriff.
- Validierung fachlicher Regeln.
- Bereitstellung strukturierter Fehlerantworten.

3.3 Datenbank

Als relationale Datenbank wird PostgreSQL vorgesehen. Primär- und Fremdschlüssel sichern eindeutige Identifikation und referenzielle Integrität. Häufig verwendete Suchfelder werden durch geeignete Indizes unterstützt.

3.4 Authentifizierung und Autorisierung

Für den Hochschulbetrieb kann LDAP beziehungsweise ein vergleichbarer zentraler Authentifizierungsdienst verwendet werden. Für lokale Entwicklung und automatisierte Tests

können Testkonten bereitgestellt werden. Nach erfolgreicher Authentifizierung werden Berechtigungen serverseitig geprüft.

3.5 Dateiverwaltung

Dateien werden serverseitig oder in einem dafür vorgesehenen Speicher abgelegt. In der Datenbank werden ausschließlich die zur Verwaltung erforderlichen Metadaten und ein technischer Verweis auf den Speicherort geführt. Zulässige Dateitypen und Maximalgrößen werden konfigurierbar validiert.

4. Funktionale Umsetzung

4.1 Benutzerverwaltung

Beim ersten erfolgreichen Login wird ein lokaler Benutzerdatensatz angelegt oder aktualisiert. Das Profil enthält nur die für die Anwendung erforderlichen Informationen. Rollen werden serverseitig ausgewertet.

4.2 Projektübersicht

Das Backend liefert sichtbare Projekte paginiert oder als begrenzte Ergebnisliste. Das Frontend stellt Titel, Kurzbeschreibung, Projektart, Fachbereich und Status dar.

4.3 Projekterstellung

Das Formular enthält mindestens Titel, Beschreibung, Projektart und Fachbereich. Projektart und Fachbereich werden über definierte Auswahlwerte dargestellt. Nach erfolgreicher Validierung wird das Projekt gespeichert.

4.4 Projektdetails

Die Detailansicht lädt Projektdaten, Teamzuordnungen und verfügbare Dokumente. Aktionen werden abhängig von Rolle und Projektzugehörigkeit angezeigt.

4.5 Einladungen

Einladungen besitzen die Statuswerte OFFEN, ANGENOMMEN, ABGELEHNT und optional ABGELAUFEN. Eine Entscheidung wird transaktional gespeichert; bei Annahme wird die Teamzuordnung erzeugt oder aktiviert.

4.6 Dokumentenmanagement

Beim Upload werden Dateityp, Größe, Projektberechtigung und Dateiname geprüft. Nach erfolgreicher Speicherung wird ein Metadatensatz angelegt. Download und Löschung erfordern eine erneute Berechtigungsprüfung.

4.7 Suche und Filter

Suchbegriffe werden auf geeignete Projektfelder angewendet. Filterwerte werden getrennt übergeben und serverseitig validiert.

4.8 Dashboard

Das Dashboard bündelt eigene Projekte, offene Einladungen und Schnellzugriffe. Die Daten können über einen aggregierten Endpunkt oder mehrere fachlich getrennte Endpunkte geladen werden.

5. Datenmodell

Das Datenmodell wird relational aufgebaut. Die zentrale Entität ist project. Benutzer werden über project_members Projekten zugeordnet. Einladungen referenzieren Projekt, Absender und Empfänger. Dokumente referenzieren Projekt und Uploader.

Tabelle	Zweck	Zentrale Felder
users	Lokale Benutzerinformationen	id, first_name, last_name, email, role_id, account_status, created_at
roles	Rollenstammdaten	id, name, description
projects	Projektstammdaten	id, title, description, project_type_id, department_id, status, created_by, created_at
project_types	Auswahlwerte Projektart	id, name, active
departments	Auswahlwerte Fachbereich	id, name, active
project_members	Zuordnung Nutzer-Projekt	id, project_id, user_id, member_role, joined_at
invitations	Projekteinladungen	id, project_id, sender_id, recipient_id, status, created_at, decided_at
documents	Dokumentenmetadaten	id, project_id, uploader_id, original_name, stored_name, mime_type, file_size, uploaded_at

6. Benutzeroberfläche

Seite	Inhalte / Funktionen
Login	Anmeldeformular, Fehlermeldung, Weiterleitung nach erfolgreicher Anmeldung.

Seite	Inhalte / Funktionen
Dashboard	Eigene Projekte, offene Einladungen, Schnellzugriffe.
Projektübersicht	Suche, Filter, Projektkarten, Navigation zu Details.
Projektdetails	Beschreibung, Status, Projektart, Fachbereich, Team, Dokumente, zulässige Aktionen.
Projekt erstellen	Formular mit Pflichtfeldern und Dropdown-Auswahl für Projektart und Fachbereich.
Einladungen	Liste offener und bearbeiteter Einladungen mit Annahme-/Ablehnungsaktion.
Profil	Anzeige und Bearbeitung freigegebener Profildaten.

6.1 Navigationskonzept

Die Hauptnavigation bleibt auf den zentralen Seiten konsistent. Aktive Bereiche werden kenntlich gemacht. Nach Aktionen wie Projekterstellung oder Einladungsentscheidung wird eine verständliche Rückmeldung angezeigt.

6.2 Formularverhalten

- Pflichtfelder werden gekennzeichnet.
- Ungültige Eingaben werden feldnah erläutert.
- Während laufender Requests werden Mehrfachübermittlungen verhindert.
- Nach erfolgreicher Speicherung werden Daten aktualisiert oder die Zielseite neu geladen.

7. Nichtfunktionale Umsetzung

7.1 Benutzerfreundlichkeit

Die Oberfläche wird mit wiederkehrenden Gestaltungsmustern umgesetzt. Schaltflächen, Formulare, Statuskennzeichnungen und Fehlermeldungen sollen auf allen Seiten konsistent verwendet werden.

7.2 Sicherheit

- Serverseitige Rechteprüfung für jede geschützte Aktion.
- Validierung aller externen Eingaben.
- Keine Speicherung von Passwörtern im Klartext.
- Beschränkung von Uploadtypen und Dateigrößen.
- Produktiver Zugriff über HTTPS.

- Keine vertrauenswürdige Autorisierung ausschließlich im Frontend.

7.3 Datenschutz

- Datenminimierung.
- Beschränkter Zugriff auf personenbezogene Daten.
- Keine realen personenbezogenen Daten in Testdatensätzen.
- Protokollierung nur im erforderlichen Umfang.

7.4 Performance

- Indizes auf häufig gefilterten und verknüpften Feldern.
- Begrenzung großer Ergebnislisten.
- Vermeidung unnötiger mehrfacher API-Aufrufe.
- Dateiinhalte werden nicht als große Binärdaten direkt in Standardlisten übertragen.

7.5 Wartbarkeit

- Klare Schichtenstruktur im Backend.
- Komponentenbasierte Struktur im Frontend.
- Zentrale Fehlerbehandlung.
- Konfiguration umgebungsabhängiger Werte außerhalb des Quellcodes.
- Versionskontrolle und Code-Reviews.

8. Schnittstellen und API

Methode	Endpunkt	Zweck
POST	/api/auth/login	Anmeldung eines Test- oder Hochschulnutzers
GET	/api/projects	Projektliste mit optionalen Such- und Filterparametern
POST	/api/projects	Neues Projekt anlegen
GET	/api/projects/{id}	Projektdetails abrufen
PUT	/api/projects/{id}	Projekt aktualisieren
GET	/api/project-types	Aktive Projektarten abrufen
GET	/api/departments	Aktive Fachbereiche abrufen
GET	/api/invitations/me	Eigene Einladungen abrufen
POST	/api/invitations	Einladung erzeugen
POST	/api/invitations/{id}/accept	Einladung annehmen
POST	/api/invitations/{id}/decline	Einladung ablehnen

Methode	Endpunkt	Zweck
POST	/api/projects/{id}/documents	Dokument hochladen
GET	/api/documents/{id}/download	Dokument herunterladen

8.1 Beispiel einer Projektantwort

Eine Projektantwort kann Felder wie id, title, description, projectType, department, status, createdBy und createdAt enthalten. Beziehungen zu Team und Dokumenten können je nach Endpunkt eingebettet oder separat geladen werden.

9. Fehlerbehandlung und Validierung

Situation	Erwartetes Verhalten
Pflichtfeld fehlt	HTTP 400 und verständliche Validierungsinformation.
Nicht angemeldet	HTTP 401.
Keine Berechtigung	HTTP 403.
Projekt nicht vorhanden	HTTP 404.
Doppelte oder bereits bearbeitete Einladung	HTTP 409 oder fachlich gleichwertige Konfliktantwort.
Nicht unterstützter Dateityp	Upload wird abgelehnt und begründet.
Interner Fehler	HTTP 500; technische Details werden nicht ungefiltert an den Client ausgegeben.

10. Projektplan

Phase	Inhalte	Ergebnis
1 Analyse	Anforderungen, Rollen, Use Cases, Datenobjekte	Abgestimmte Dokumentation
2 Grundsystem	Frontend/Backend-Projekt, Datenbank, Basiskonfiguration	Startfähige Anwendung
3 Kernfunktionen	Projektübersicht, Projektanlage, Details	Nutzbarer Projektworkflow

Phase	Inhalte	Ergebnis
4 Team & Einladungen	Einladungstatus, Teamzuordnung	Nutzbarer Teamworkflow
5 Dokumente	Upload, Download, Metadaten	Projektbezogene Ablage
6 Qualität	Tests, Fehlerbehebung, Rechteprüfung	Stabiler Teststand
7 Abschluss	Deployment, Dokumentation, Präsentation	Abnahmefähige Version

11. Testkonzept

Testfall	Aktion	Erwartetes Ergebnis
T-01	Login mit gültigem Testkonto	Dashboard wird angezeigt.
T-02	Login mit ungültigen Daten	Zugriff wird abgelehnt; verständliche Fehlermeldung.
T-03	Projekt mit gültigen Pflichtfeldern anlegen	Projekt wird gespeichert und ist auffindbar.
T-04	Projekt ohne Titel anlegen	Speicherung wird verhindert.
T-05	Projektart/Fachbereich auswählen	Nur definierte Auswahlwerte werden übernommen.
T-06	Einladung annehmen	Status wird ANGENOMMEN; Teamzuordnung entsteht.
T-07	Einladung ablehnen	Status wird ABGELEHNT; keine Teamzuordnung.
T-08	Zulässiges PDF hochladen	Datei und Metadaten werden gespeichert.
T-09	Unzulässige Datei hochladen	Upload wird abgewiesen.
T-10	Geschützte Aktion ohne Berechtigung	Server verweigert Zugriff.
T-11	Projektfilter verwenden	Ergebnis entspricht den gewählten Kriterien.
T-12	Responsive Darstellung	Kernseiten bleiben auf

Testfall	Aktion	Erwartetes Ergebnis
		kleiner Bildschirmbreite bedienbar.

12. Abnahmekriterien

- Alle Muss-Anforderungen des Lastenhefts sind implementiert oder nachvollziehbar als abgestimmte Abweichung dokumentiert.
- Die definierten Kern-Testfälle sind erfolgreich ausführbar.
- Projektanlage, Projektanzeige, Einladungsentscheidung und Dokumentenupload funktionieren im integrierten System.
- Serverseitige Berechtigungsprüfungen verhindern unzulässige Aktionen.
- Es bestehen keine bekannten kritischen Fehler, die die Kernnutzung verhindern.

13. Deployment und Betrieb

Für den Testbetrieb werden Frontend, Backend und Datenbank in einer definierten Umgebung gestartet. Konfigurationswerte wie Datenbankzugang, Dateipfad und Authentifizierungsparameter werden umgebungsabhängig bereitgestellt. Vor einem produktiven Einsatz sind HTTPS, Backups, Logging und Wiederherstellungsverfahren verbindlich zu konfigurieren.

14. Qualitätssicherung

- Code-Reviews vor Zusammenführung größerer Änderungen.
- Automatisierte Tests für zentrale Service- und Repositorylogik.
- Integrationstests für REST-Endpunkte und Datenbankzugriffe.
- Manuelle UI-Tests für die wichtigsten Nutzerabläufe.
- Nachvollziehbare Fehlererfassung und Priorisierung.
- Abgleich von Implementierung und Dokumentation vor Abgabe.

15. Glossar

Begriff	Bedeutung
REST-API	HTTP-basierte Schnittstelle zwischen Frontend und Backend.
React	Bibliothek zur komponentenbasierten Entwicklung von Weboberflächen.
Spring Boot	Framework für Java-basierte

Begriff	Bedeutung
	Serveranwendungen.
PostgreSQL	Relationales Datenbankmanagementsystem.
LDAP	Protokoll für Verzeichnisdienste und zentrale Authentifizierung.
DTO	Datenobjekt für den Austausch zwischen Anwendungsschichten oder Schnittstellen.
RBAC	Rollenbasierte Zugriffskontrolle.
HTTP-Statuscode	Standardisierte Kennzeichnung des Ergebnisses einer HTTP-Anfrage.